

16-831: Introduction to Robot Learning, Fall 2024

Inverse Reinforcement Learning



Image from Kris Kitani

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Project

☐ Please start working on it!

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☒ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline

- ❑ Recap: Online RL
- ❑ Recap: Offline RL
- ❑ What is IRL and why IRL?
- ❑ Linear reward formulations and algorithms:
 - Linear programming IRL
 - Maximum margin IRL
 - Maximum entropy IRL

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

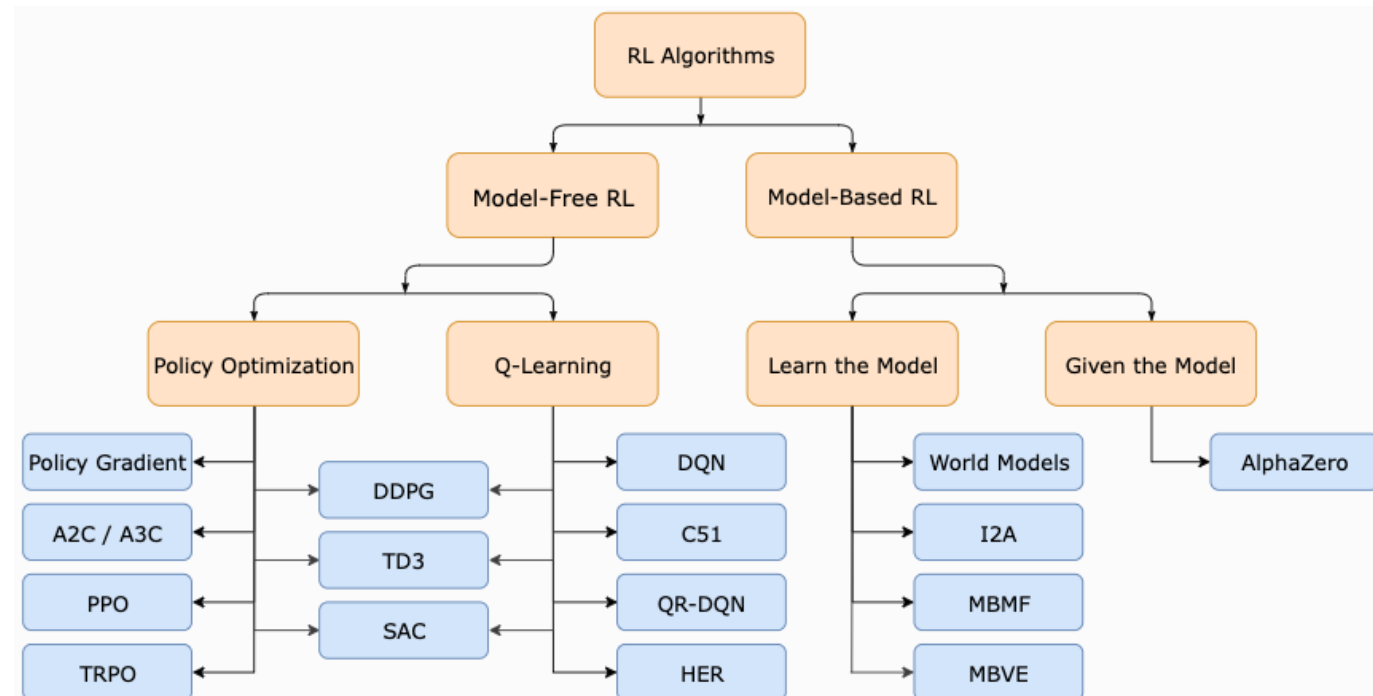
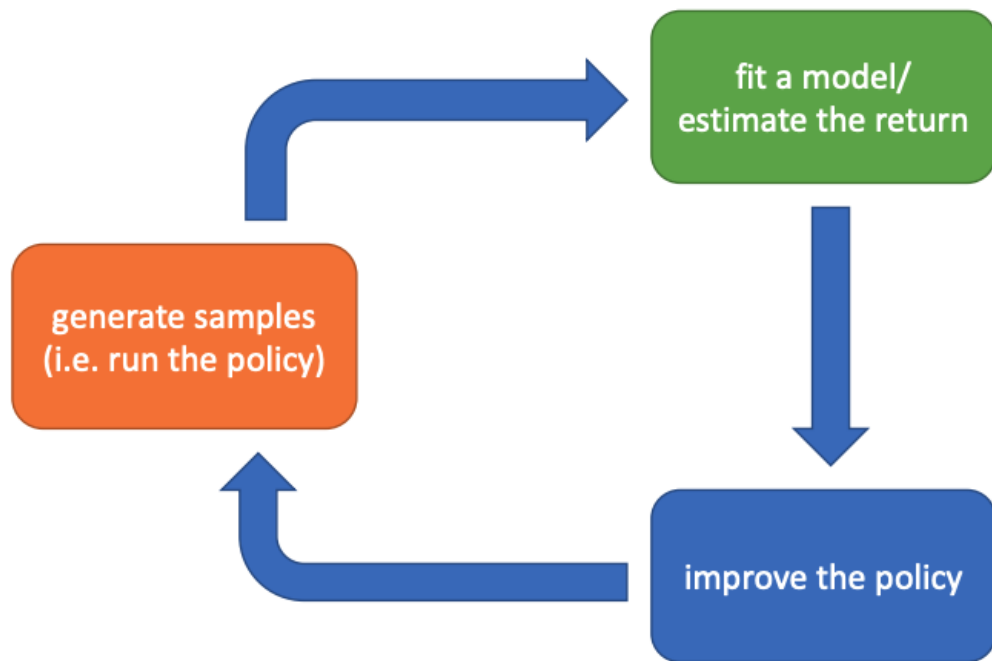
Outline

- ❑ Recap: Online RL
- ❑ Recap: Offline RL
- ❑ What is IRL and why IRL?
- ❑ Linear reward formulations and algorithms:
 - Linear programming IRL
 - Maximum margin IRL
 - Maximum entropy IRL

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Taxonomy of Online MFRL Algorithms

- ❑ **Value-based** (example: FQI, DQN): estimate value function or Q-function of the optimal policy (no explicit policy).
- ❑ **Policy-gradient** (example: REINFORCE, NPG): directly optimize the policy
- ❑ **Actor-critic** (example: SAC, DDPG): estimate value function or Q-function of the current policy, use it to improve policy



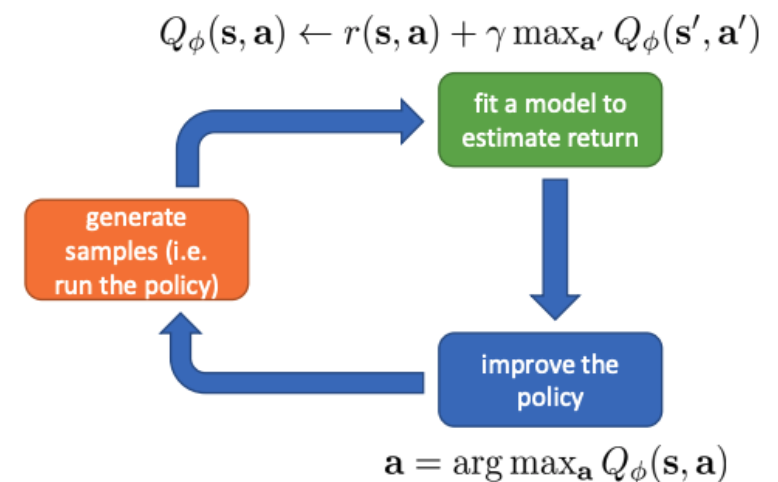
Recap: Q-Learning

□ Based on: **Bellman optimality equation**

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

□ Example: DQN (Deep Q-Network):

- Epsilon greedy policy gives a and observe (s, a, s', r)
- Update the replay buffer
- Sample a batch from the buffer. Compute the target $y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_{\phi^-}(s'_i, a'_i)$
- Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$
- Periodically update ϕ^- : $\phi^- \leftarrow \phi$



Recap: Policy Gradient

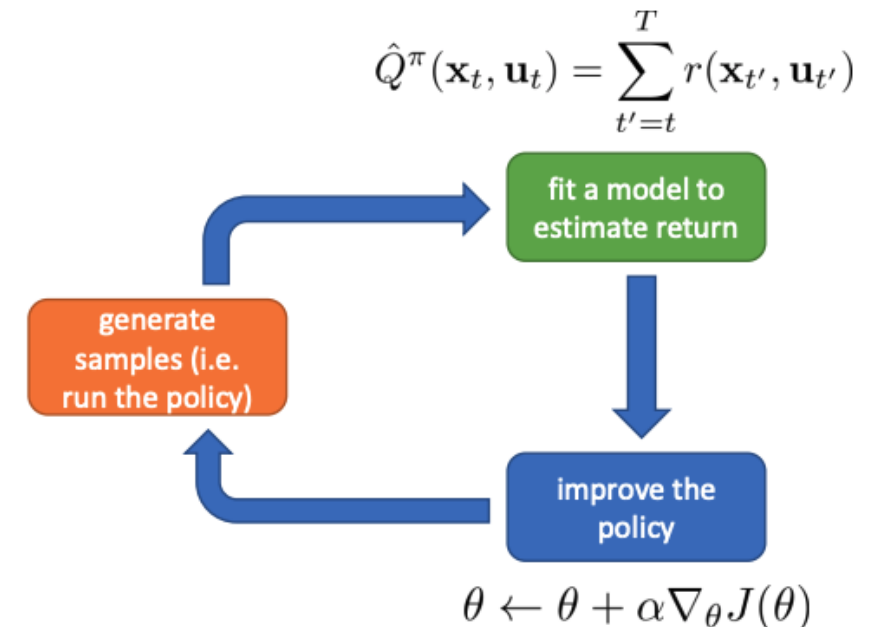
□ Based on: **Policy gradient theorem**

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$
$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)]$$

□ Example: REINFORCE

- 
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
 2. $\nabla_{\theta} J(\theta) \approx \sum_i (\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i)) (\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i))$
 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



Recap: Actor-Critic

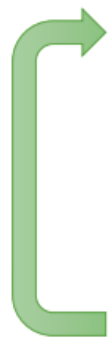
□ Based on: **Policy gradient with baselines**

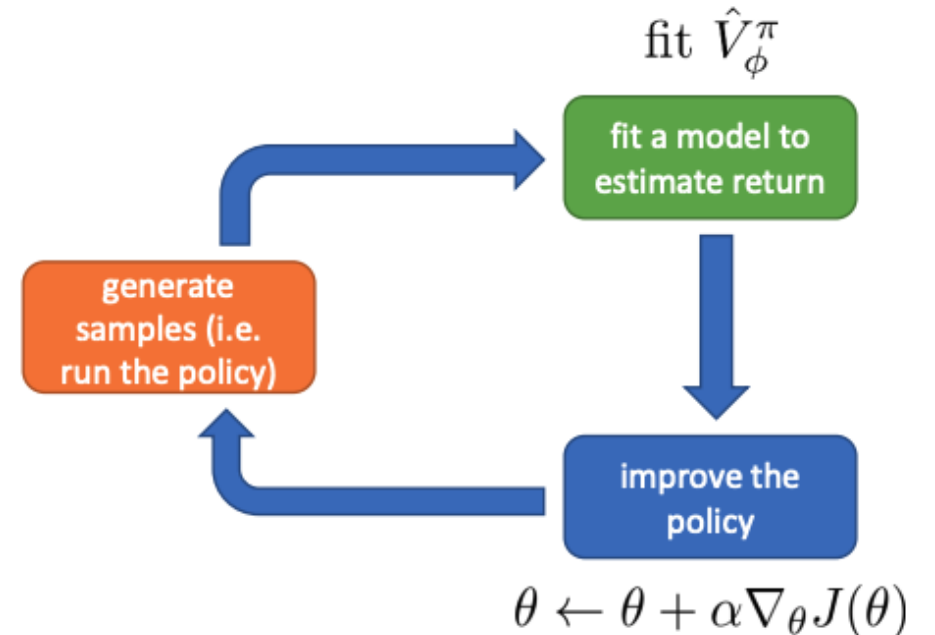
$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

- Learn $V_{\phi}^{\pi}(s)$
- Estimate $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ by $r + \gamma V_{\phi}^{\pi}(s') - V_{\phi}^{\pi}(s)$

□ Example: online actor-critic

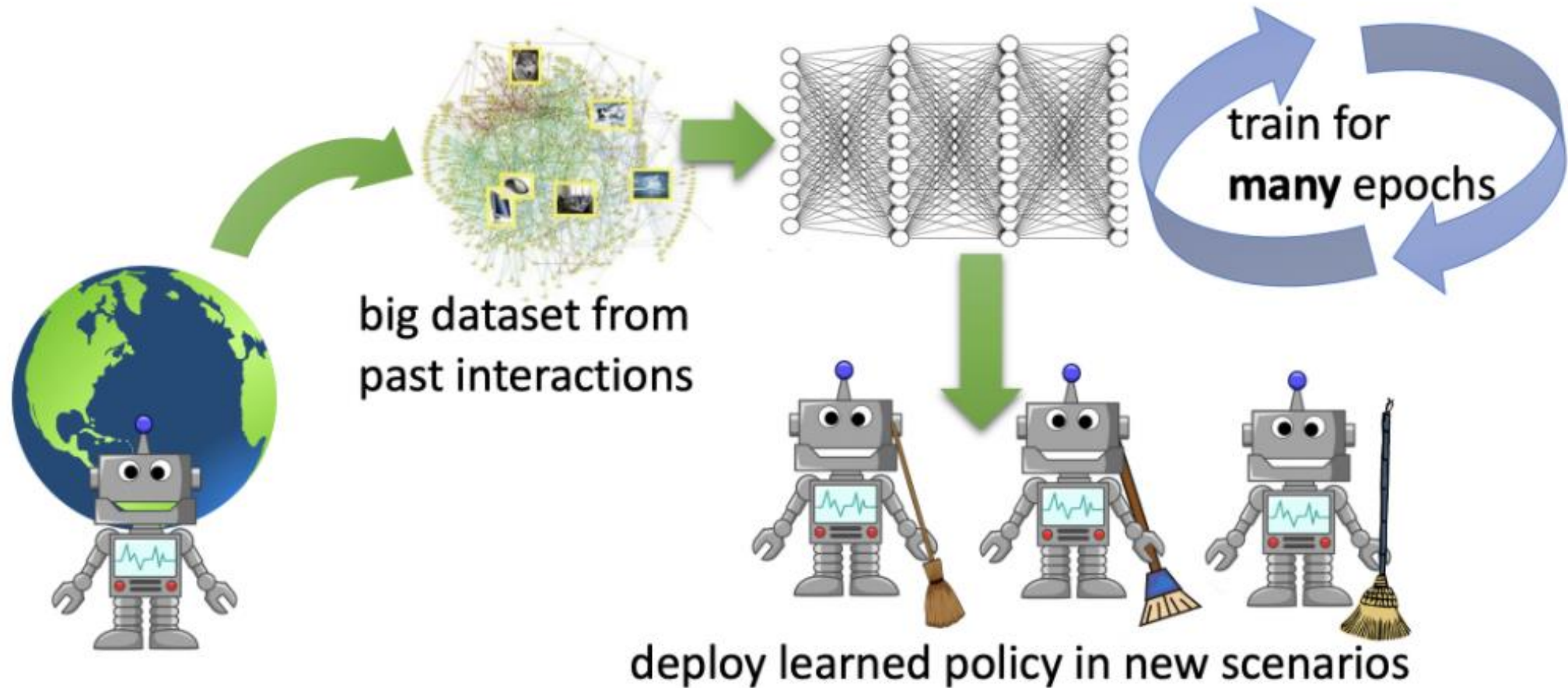
online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_{\phi}^{\pi}$ using target $r + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}')$
 3. evaluate $\hat{A}^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}') - \hat{V}_{\phi}^{\pi}(\mathbf{s})$
 4. $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}^{\pi}(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



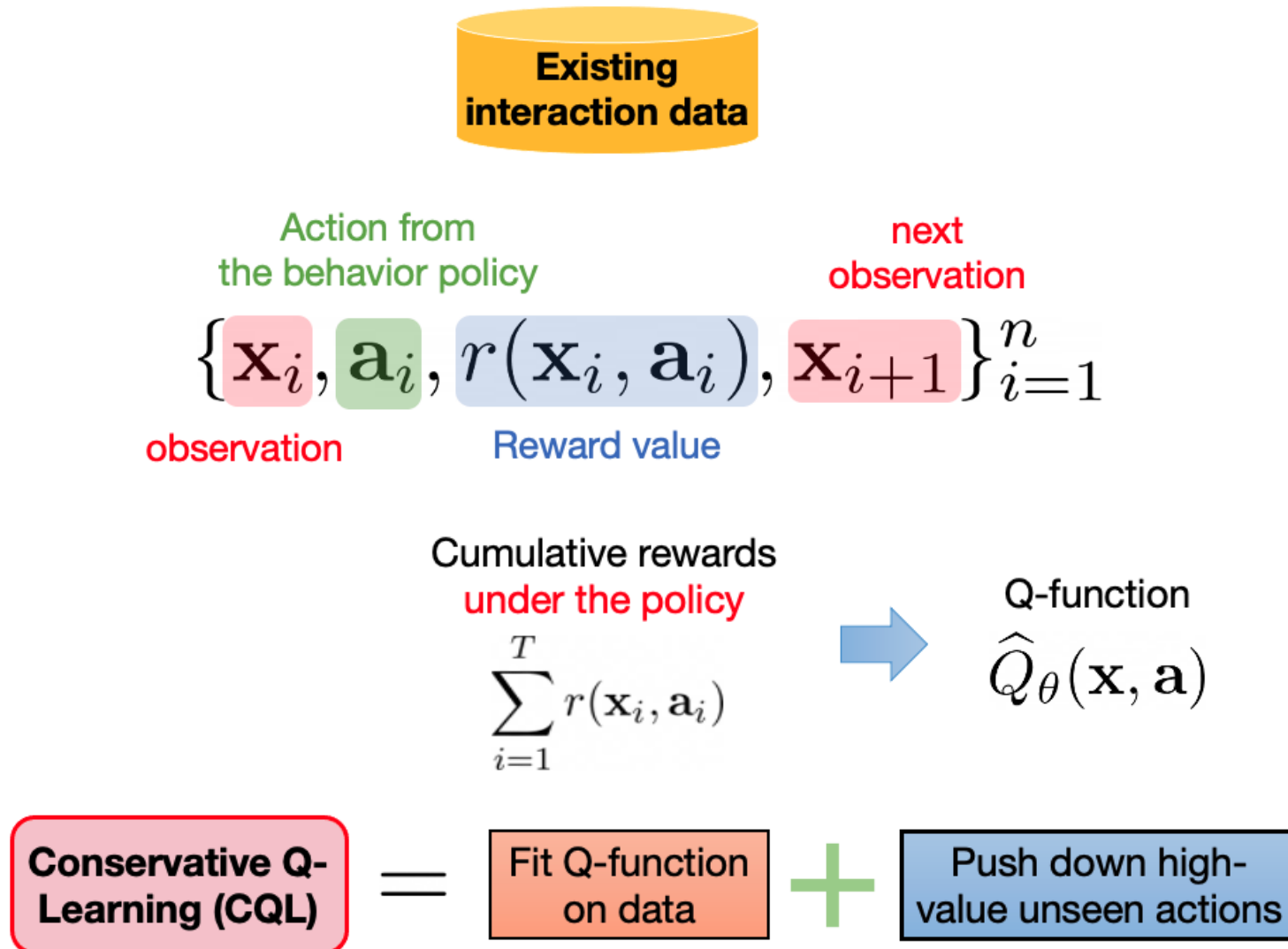
Learning from Offline Data: Offline RL

□ Recap: last lecture from Aviral Kumar



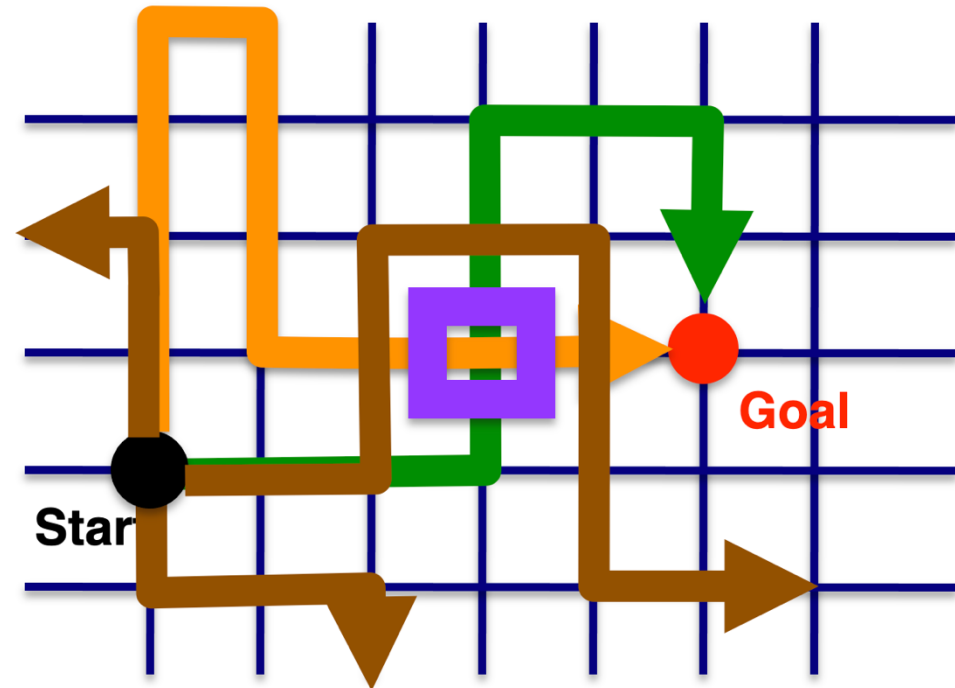
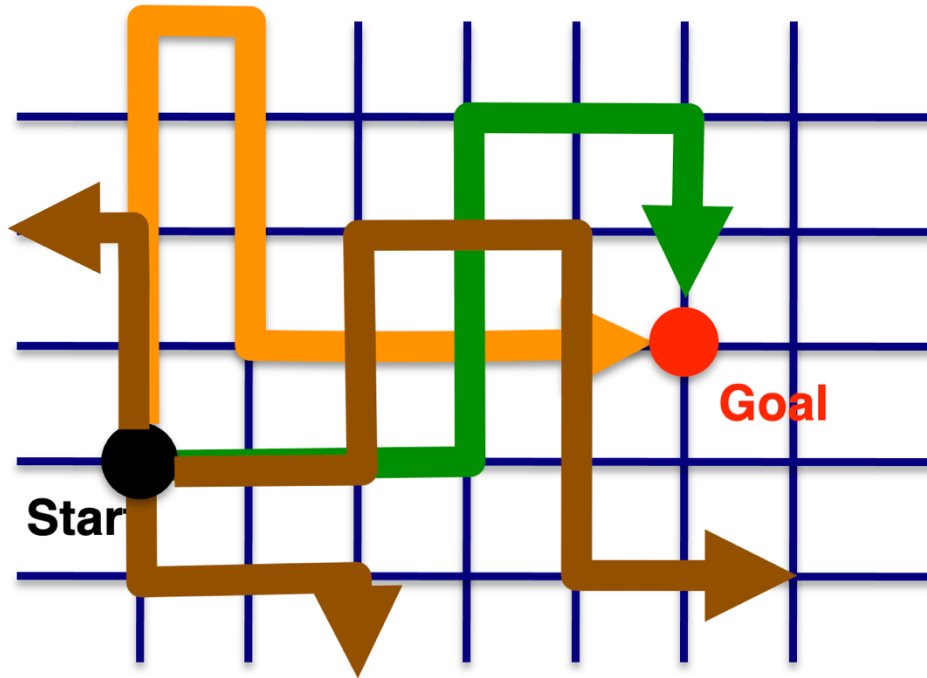
Learning from Offline Data: Offline RL

- Example: Conservative Q-learning (CQL)



Learning from Offline Data: Offline RL

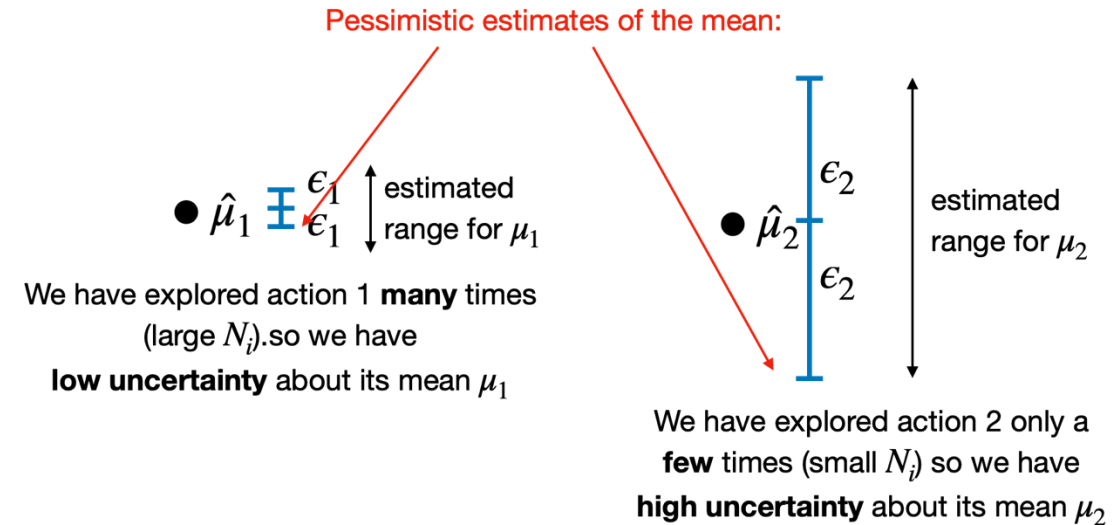
- ❑ Why is offline RL different from imitation learning (IL)?
 - Short answer: It can learn from demonstrations of mixed quality
 - "Formal" answer: Stitching



Algorithmic Difference between Online/Offline RL

❑ In online RL/bandit: being optimistic

- Recap: Upper Confidence Bounds (UCB) in bandit with regret guarantees
- Optimism in the face of uncertainty (OFU)

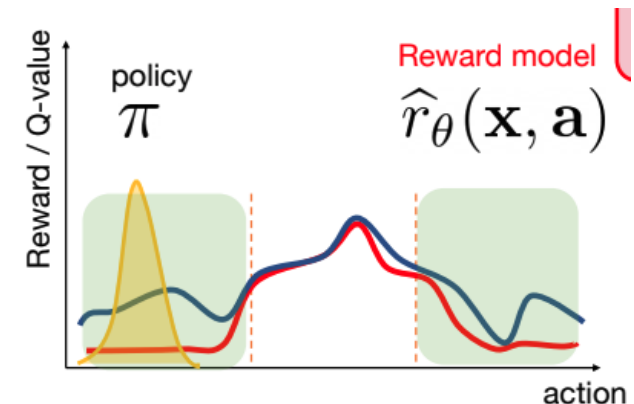


❑ In offline RL/bandit: being pessimistic

- Pessimism in the face of uncertainty

❑ Why?

- In online RL, the agent can actively and strategically explore
- In offline RL, we only have existing interaction data



Outline

- ❑ Recap: Online RL
- ❑ Recap: Offline RL
- ❑ What is IRL and why IRL?
- ❑ Linear reward formulations and algorithms:
 - Linear programming IRL
 - Maximum margin IRL
 - Maximum entropy IRL

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Inverse Reinforcement Learning (IRL)

- ❑ Goal: learning from “good” demonstrations
- ❑ I.e., a special case of imitation learning (IL)
 - Compared to offline RL: learning from “suboptimal” or “mixed” demonstrations
- ❑ No reward available (same setting as other IL algorithms such as behavior cloning)!
- ❑ Give a dataset from some expert $\pi^*: D = \{(s_i, a_i, s'_i)\}_i$
- ❑ General recipe (may need to iterate the following steps):
 - Learn a reward function r^* such that

$$\pi^* = \operatorname{argmax}_{\pi \in \Pi} \mathbb{E}_{\pi, p}[r^*(s, a)]$$

- Learn policy given the reward (an RL problem)
- Compare the learned policy with expert

Why IRL?

- Give a dataset from some expert $\pi^*: D = \{(s_i, a_i, s'_i)\}_i$
- General recipe (may need to iterate the following steps):

- Learn a reward function r^* such that

$$\pi^* = \operatorname{argmin}_{\pi \in \Pi} \mathbb{E}_{\pi, p}[r^*(s, a)]$$

- Learn policy given the reward (an RL problem)
- Compare the learned policy with expert

- Since we have demonstrations from π^* , why don't we directly learn a policy?
 - I.e., why don't we use BC?
- Intuition: Assumes learning r^* is statistically easier than directly learning π^*

Why IRL?

Types of Imitation Learning

Behavioral Cloning

$$\operatorname{argmin}_{\theta} E_{(s,a^*) \sim P^*} L(a^*, \pi_{\theta}(s))$$

Works well when P^* close to P_{θ}

Inverse RL

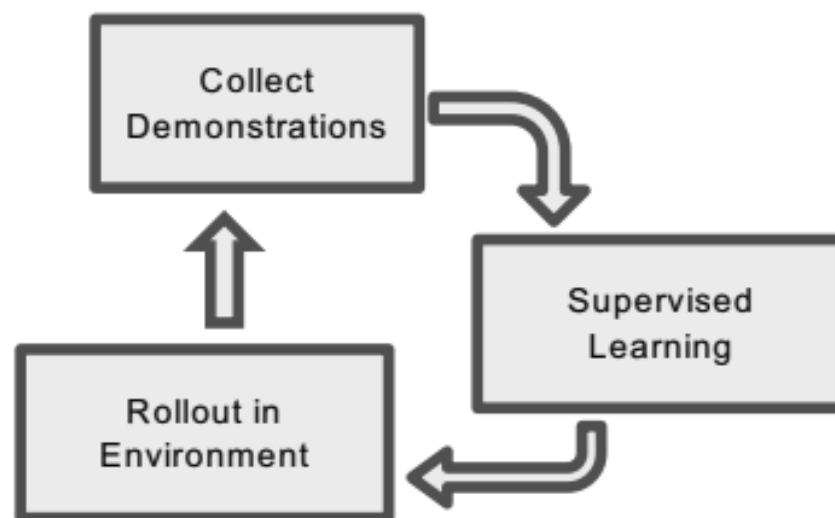
Learn r such that:

$$\pi^* = \operatorname{argmax}_{\theta} E_{s \sim P(s|\theta)} r(s, \pi_{\theta}(s))$$

RL problem

Assumes learning r is statistically easier than directly learning π^*

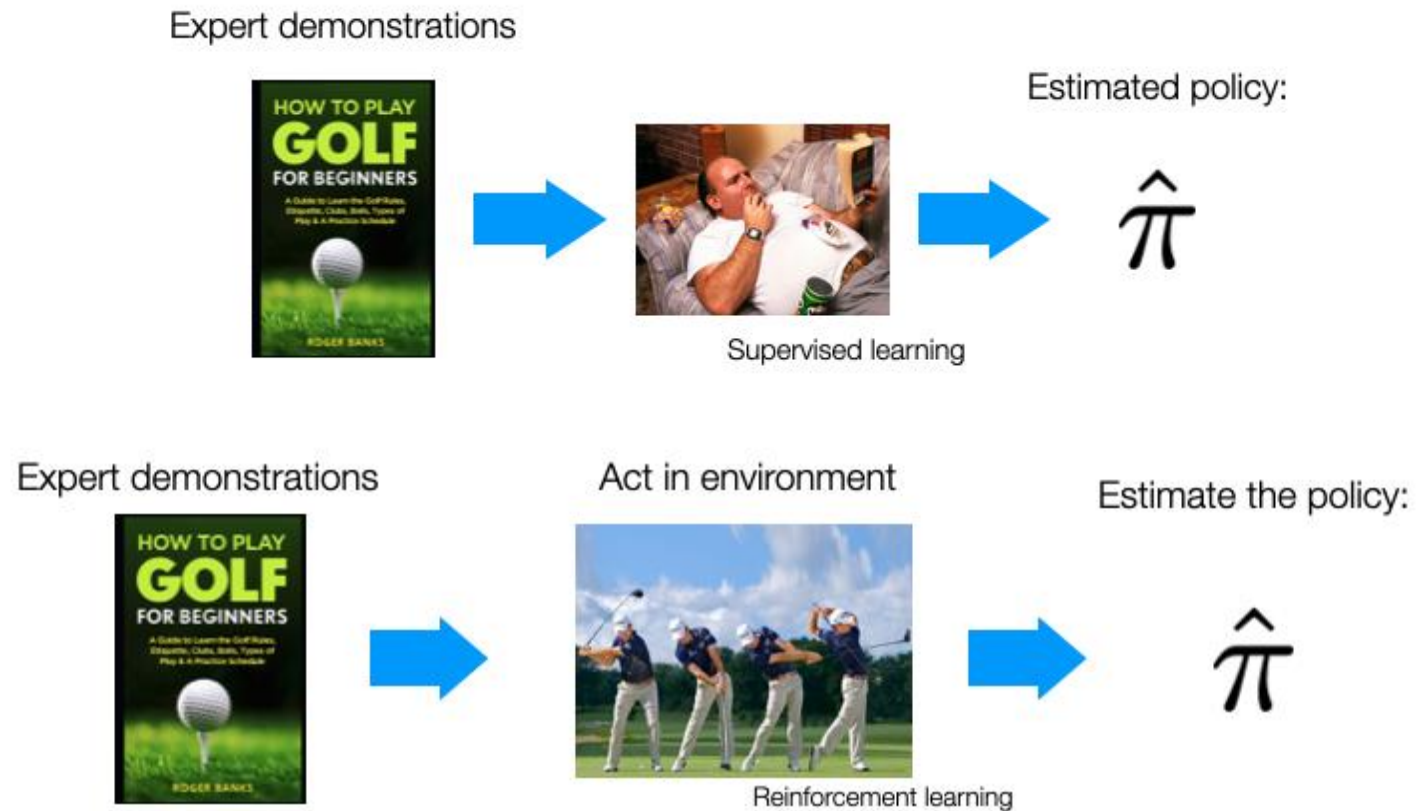
Direct Policy Learning via Interactive Demonstrator



**Requires Interactive Demonstrator
(BC is 1-step special case)**

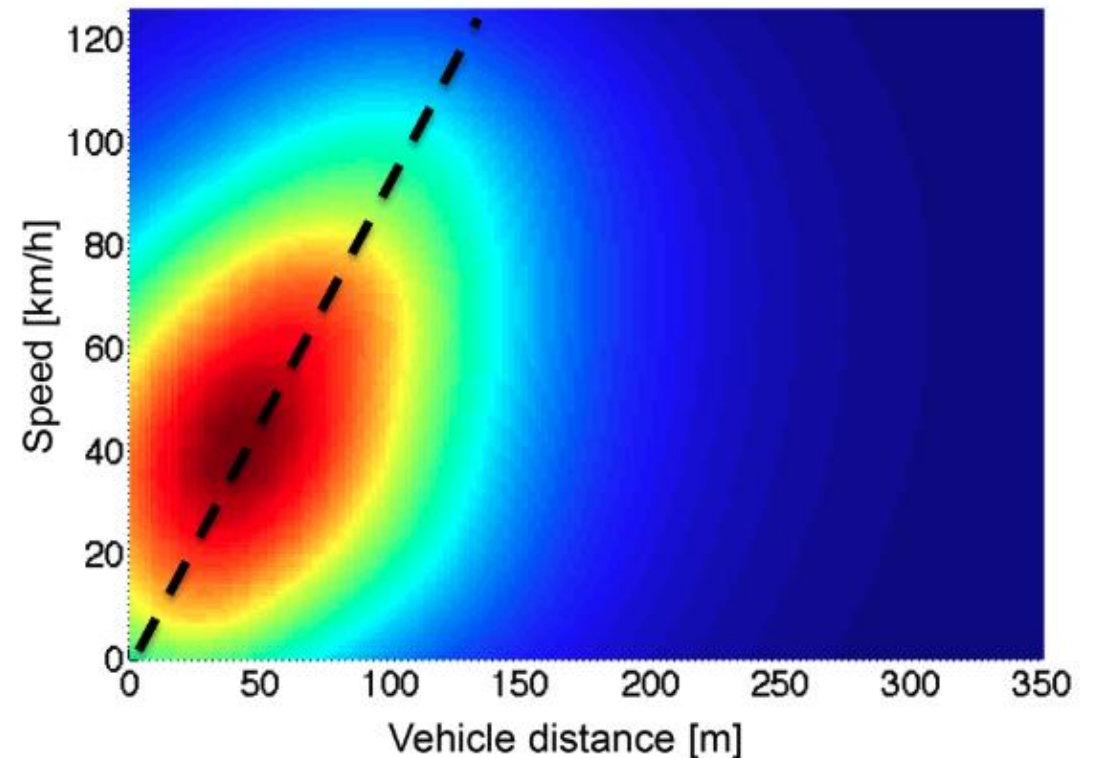
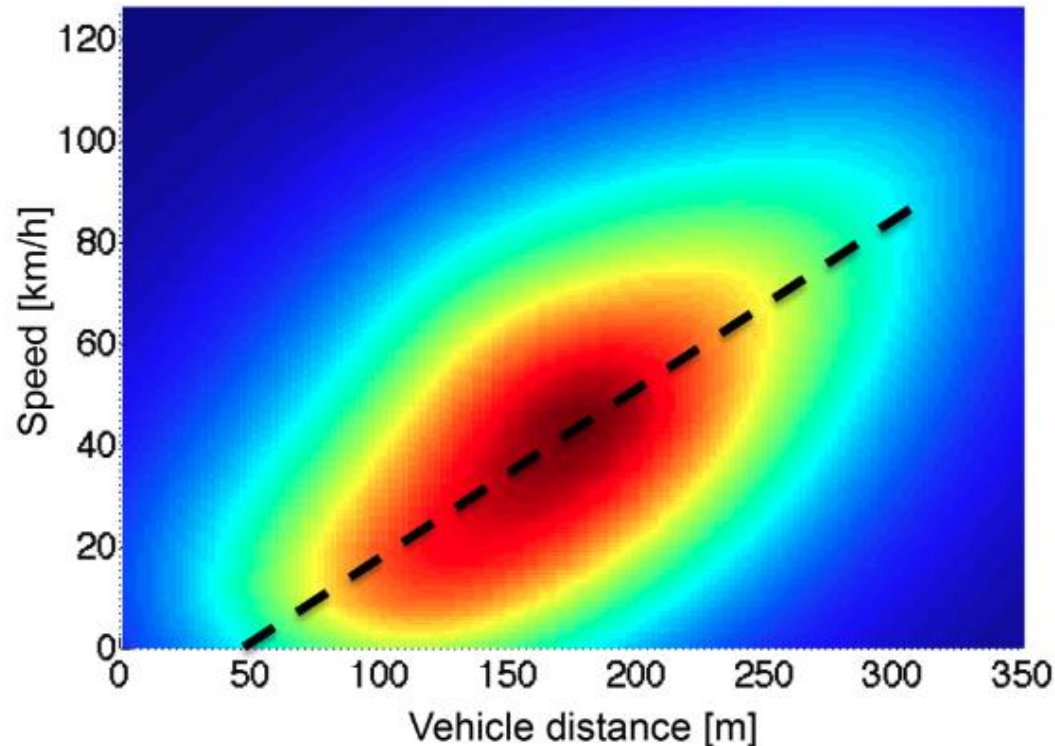
Why IRL?

- ❑ (Perhaps) closer to how human learns
- ❑ Learning how to play golf:
 - Top: BC
 - Below: IRL



Why IRL?

- ❑ Advantages of learning a reward function:
 - More interpretable
 - Can generalize or adapt to new environments via RL (once we have a reward)
- ❑ Disadvantages: typically need extra RL steps for policy optimizations
- ❑ Example: reward functions from two different drivers in car following problems



Outline

- ❑ Recap: Online RL
- ❑ Recap: Offline RL
- ❑ What is IRL and why IRL?
- ❑ Linear reward formulations and algorithms:
 - Linear programming IRL
 - Maximum margin IRL
 - Maximum entropy IRL

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Linear Reward Formulations

- ❑ Commonly used in IRL algorithms
- ❑ Easier to optimize
- ❑ θ : unknown weight; ϕ : known features (e.g., DNN)

$$R_{\theta}(s) = \theta_1 \phi_1(s) + \cdots + \theta_d \phi_d(s) = \boldsymbol{\theta}^{\top} \boldsymbol{\phi}(s)$$

- ❑ Why linear?
- ❑ Value function is linear in “expected features” $\mu^{\pi}(s)$!

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t r(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\theta} \cdot \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \boldsymbol{\mu}^{\pi}(s) \end{aligned}$$

$$V^{\pi}(s) = \boldsymbol{\theta}^{\top} \boldsymbol{\mu}^{\pi}(s)$$

Linear Reward Formulations

□ Value function is linear in “expected features” $\mu^\pi(s)$!

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t r(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\theta} \cdot \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s^{(t)}) \mid s^{(0)} = s \right] \\ &= \boldsymbol{\theta} \cdot \boldsymbol{\mu}^\pi(s) \end{aligned} \quad V^\pi(s) = \boldsymbol{\theta}^\top \boldsymbol{\mu}^\pi(s)$$

□ Can estimate from data (Monte Carlo):

$$\begin{aligned} \boldsymbol{\mu}^\pi(s) &= \mathbb{E}_p \left[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{\phi}(s^{(t)}) \right] \\ &\approx \frac{1}{M} \sum_{m=1}^M \sum_{t=0}^{T_m} \gamma^t \boldsymbol{\phi}(s_m^{(t)}) \end{aligned}$$

Linear Programming IRL (LP-IRL)

- ❑ Key idea: maximize the difference between the expert and other policies π_n
- ❑ How to get other policies? RL, manually designed, etc
- ❑ We can efficiently compute V^π by linearly mixing its expected features $\mu^\pi(s)$! For both expert and other policies
- ❑ This process can be iterative: solve θ , train new π_n (through RL with the learned reward $\theta^\top \phi$), solve new θ , ...

**Linear Program
IRL**

[Ng & Russell 2000]

$$\begin{aligned} \hat{\theta} = \arg \max_{\theta} & \left\{ \sum_n \hat{V}^{\pi^*}(s_0) - \hat{V}^{\pi_n}(s_0) \right\} \\ \text{s.t. } & |\theta_d| \leq 1 \quad \forall d \end{aligned}$$

Algorithms for Inverse Reinforcement Learning

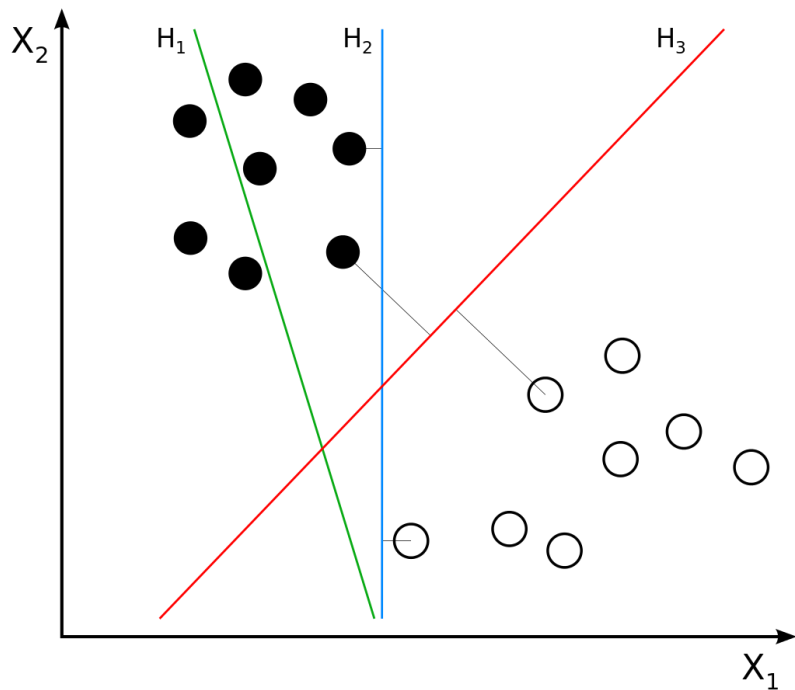
Andrew Y. Ng
Stuart Russell

Computer Science Division, U.C. Berkeley, Berkeley, CA 94720 USA

ANG@CS.BERKELEY.EDU
RUSSELL@CS.BERKELEY.EDU

Max Margin IRL (Apprenticeship Learning)

- ❑ By Pieter Abbeel and Andrew Ng (ICML 2004)
- ❑ Similar ideas, but adopt the **max margin philosophy** from SVM (support vector machine)



Max Margin IRL

[Abbeel & Ng 2004]

$$\begin{aligned} \min_{\theta} \quad & \lambda \|\theta\|_2 + \sum_i \xi_i \\ \text{s.t.} \quad & \theta^\top (\mu(\pi^*) - \mu(\pi_i)) \geq (1 - \xi_i) \quad \forall i \end{aligned}$$

Apprenticeship Learning via Inverse Reinforcement Learning

Pieter Abbeel
Andrew Y. Ng

Computer Science Department, Stanford University, Stanford, CA 94305, USA

PABBEEL@CS.STANFORD.EDU
ANG@CS.STANFORD.EDU

Max Margin IRL (Apprenticeship Learning)

- ❑ Applications in autonomous helicopter acrobatics
- ❑ Abbeel, Coates, Ng, IJRR 2010

Autonomous Helicopter Aerobatics through Apprenticeship Learning

Pieter Abbeel¹, Adam Coates² and Andrew Y. Ng²



The International Journal of
Robotics Research
29(13) 1608–1639
© The Author(s) 2010
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/0278364910371999
ijr.sagepub.com
SAGE



Maximum Entropy IRL (MaxEnt IRL)

- ❑ IRL is an “ill-posed” problem:
 - Many reward functions correspond to the same policy
 - Many stochastic mixtures of policies correspond to the same feature expectation
- ❑ Need to introduce some regularization
- ❑ Maximum entropy principle!

$$\max_P - \sum_{\tau} P(\tau) \log P(\tau)$$

Maximum entropy principle: The probability distribution which best represents the current state of knowledge is the one with largest entropy (E.T. Jaynes 1957)

Information Theory and Statistical Mechanics - E.T. Jaynes - Phys. Rev. 1957

- ❑ Occam's Razor: “The simpler explanation is to be preferred.”
- ❑ The distribution with largest entropy (uncertainty) makes the least assumptions about the true distribution

Maximum Entropy IRL (MaxEnt IRL)

□ From CMU: Ziebart, Maas, Bagnell, Dey, AAAI 2008

Maximum Entropy Inverse Reinforcement Learning

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

bziebart@cs.cmu.edu, amaas@andrew.cmu.edu, dbagnell@ri.cmu.edu, anind@cs.cmu.edu

□ In the linear case, closed-form MaxEnt formulation:

$$P(\tau|\theta) = \frac{1}{Z(\theta)} e^{\sum_{s_t \in \tau} \theta^\top \phi(s_t)}$$



$$Z(\theta) = \int e^{r(\tau|\theta)} d\tau$$

□ Then learning reward via maximizing the log likelihood (same as generative models!)

$$\theta^* = \operatorname{argmax}_{\theta} L(\theta) = \operatorname{argmax}_{\theta} \sum_{\tau_i \in \mathcal{D}} \log P(\tau_i|\theta)$$

Maximum Entropy IRL (MaxEnt IRL)

❑ From CMU: Ziebart, Maas, Bagnell, Dey, AAAI 2008

Maximum Entropy Inverse Reinforcement Learning

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey

School of Computer Science

Carnegie Mellon University

Pittsburgh, PA 15213

bziebart@cs.cmu.edu, amaas@andrew.cmu.edu, dbagnell@ri.cmu.edu, anind@cs.cmu.edu

❑ Then learning reward via maximizing the log likelihood (same as generative modelling!)

$$\theta^* = \operatorname{argmax}_{\theta} L(\theta) = \operatorname{argmax}_{\theta} \sum_{\tau_i \in \mathcal{D}} \log P(\tau_i | \theta)$$

❑ Ziebart et al. provided an efficient way to compute the gradient of the log likelihood!

Maximum Entropy IRL (MaxEnt IRL)

□ Maximum entropy principles in RL and control:

- Can be viewed as “softmax”

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | \mathbf{s}_t) \parallel \frac{\exp(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot))}{Z^{\pi_{\text{old}}}(\mathbf{s}_t)} \right). \quad (4)$$

Soft Actor-Critic (MFRL)

$$P(\tau | \theta) = \frac{1}{Z(\theta)} e^{\sum_{s_t \in \tau} \theta^\top \phi(s_t)}$$


 $Z(\theta) = \int e^{r(\tau|\theta)} d\tau$

MaxEnt IRL

for $i \leftarrow 0$ **to** $N - 1$ **do**

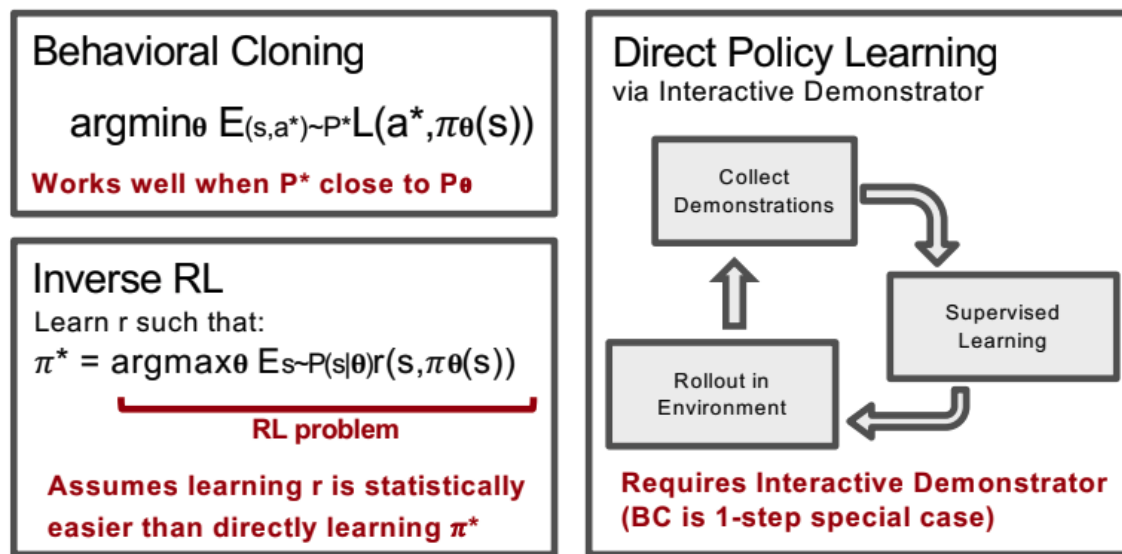
$$\begin{bmatrix} \mathbf{u}_i = \\ \mathbf{u}_i + \mathcal{H}^{-1} \mathcal{G} \left[\sum_{k=1}^K \left(\frac{\exp(-\frac{1}{\lambda} \tilde{S}(\tau_k)) \frac{\epsilon_{j,k}}{\sqrt{\Delta t}}}{\sum_{k=1}^K \exp(-\frac{1}{\lambda} \tilde{S}(\tau_k))} \right) \right]; \end{bmatrix}$$

MPPI (MBRL)

Summary: How to Learn from Demonstrations?

- ❑ Learning from “good” demonstrations: imitation learning (no need for reward)

Types of Imitation Learning



- ❑ Learning from “mixed” demonstrations: offline RL (needs reward)
- ❑ Other methods to learn from demonstrations: MBRL

Thank You!